```sql
-- #1: List names and sellers of products that are no longer available (quantity=0)
select merchants.name, products.name
from merchants
join sell using (mid)
join products using (pid)
where quantity_available = 0;
-- Lists all products that are out of stock by joining merchants, sell, and products
-- and filtering for quantity_available = 0

```

100% 30:6 2 errors found

Result Grid Filter Rows: Search Export:

name	name
Acer	Router
Acer	Network Card
Apple	Printer
Apple	Router
HP	Laptop
HP	Router
HP	Super Drive
Dell	Router
Lenovo	Ethernet Adapter

```
10    -- #2: List names and descriptions of products that are not sold.
11    select name, description
12    from products
13    except
14    select name, description
15    from products
16    join sell using (pid);
```

00% 23:16 2 errors found

Result Grid Filter Rows: Search Export:

name	description
Super Drive	External CD/DVD/RW
Super Drive	UInternal CD/DVD/RW

```
20    select name, description
21    from products
22    left outer join sell using (pid)
23    where sell.pid is null;
24    -- Retrieves products with no record in sell by using a left outer join or except
25
26    -- #3: How many customers bought SATA drives but not any routers?
```

00% 23:23 2 errors found

Result Grid Filter Rows: Search Export:

name	description
Super Drive	External CD/DVD/RW
Super Drive	UInternal CD/DVD/RW

```
-- #3: How many customers bought SATA drives but not any routers?
select count(*)
from (
    select distinct place.cid
    from place
    join orders using (oid)
    join contain using (oid)
    join products using (pid)
    where products.name = 'Hard Drive'
    and products.description like '%SATA%'
    except
    select distinct place.cid
    from place
    join orders using (oid)
    join contain using (oid)
    join products using (pid)
    where products.name = 'Router'
) as sata_no_router;
-- Counts distinct customers who bought SATA drives but never any route

```

100% 26:40 2 errors found

Result Grid Filter Rows: Search Export:

count(*)
0

```sql
-- #4: HP has a 20% sale on all its Networking products.
select products.name, sell.price as regular_price, sell.price * 0.8 as sale_price
from products
join sell using (pid)
join merchants using (mid)
where merchants.name = 'HP' and products.category = 'Networking';
-- Shows HP Networking products with a 20% discounted price by joining merchants, sell, and products
-- and calculating sell.price * 0.8
```

100% 27:50 2 errors found

Result Grid Filter Rows: Search Export:

name	regular_price	sale_price
Router	1034.46	827.5680000000001
Network Card	1154.68	923.7440000000001
Network Card	345.01	276.008
Network Card	262.2	209.76
Ethernet Adapter	1260.45	1008.3600000000001
Router	205.56	164.448
Router	1474.87	1179.896
Router	552.02	441.616
Router	100.95	80.76
Network Card	1179.01	943.2080000000001

```sql
-- #5: What did Uriel Whitney order (make sure to at least retrieve product names and prices).
select distinct products.name, merchants.name as seller, sell.price
from customers
join place using (cid)
join orders using (oid)
join contain using (oid)
join products using (pid)
join sell using (pid)
join merchants using (mid)
where customers.fullname = 'Uriel Whitney';
-- Lists all products ordered by Uriel Whitney with all their sellers and price
-- by joining customers, place, orders, contain, products, sell, and merchants
```

name	seller	price
Monitor	Acer	1435.38
Router	Acer	521.07
Network Card	Acer	130.43
Super Drive	Acer	1015.95
Super Drive	Acer	356.13
Router	Acer	1256.57
Desktop	Acer	311.06
Hard Drive	Acer	1151.28
Router	Acer	780.65
Network Card	Acer	609.2
Super Drive	Acer	671.75
Ethernet Ad…	Acer	446.62
Printer	Acer	836.28
Network Card	Acer	837.12
Printer	Acer	310.83
Hard Drive	Acer	836.99
Printer	Acer	1345.37
Laptop	Acer	33.5
Router	Acer	394.04
Laptop	Acer	247.96
Hard Drive	Acer	333.71
Monitor	Acer	1103.47
Laptop	Acer	522.73
Network Card	Acer	405.4
Router	Acer	945.51
Super Drive	Acer	1124.26
Super Drive	Acer	1135.3
Monitor	Apple	573.31
Router	Apple	979.27
Super Drive	Apple	373.31
Super Drive	Apple	1406.52
Router	Apple	575.75
Desktop	Apple	883.55
Monitor	Apple	993.63
Hard Drive	Apple	935.66
Router	Apple	1328.35
Network Card	Apple	108.3
Ethernet Ad…	Apple	878.23
Printer	Apple	721.09
Network Card	Apple	1291.8
Network Card	Apple	791.7
Printer	Apple	994.35
Hard Drive	Apple	1328.19
Printer	Apple	397.92
Laptop	Apple	739.02
Laptop	Apple	379.06
Hard Drive	Apple	773.55
Monitor	Apple	786.31

	Laptop	Apple	425.1	
	Super Drive	Apple	540.56	
	Network Card	Apple	925.57	
	Router	Apple	895.76	
	Super Drive	Apple	766.96	
	Super Drive	Apple	766.67	
	Monitor	HP	822.33	
	Router	HP	1034.46	
	Network Card	HP	1179.01	
	Super Drive	HP	658.52	
	Router	HP	205.56	
	Desktop	HP	1490.37	
	Monitor	HP	991.41	
	Hard Drive	HP	939.55	
	Router	HP	1474.87	
	Network Card	HP	262.2	
	Super Drive	HP	280.91	
	Ethernet Ad…	HP	1260.45	
	Printer	HP	358.01	
	Network Card	HP	345.01	
	Network Card	HP	1154.68	
	Printer	HP	1408.8	
	Printer	HP	856.22	
	Laptop	HP	585.27	
	Router	HP	100.95	
	Laptop	HP	120.44	
	Hard Drive	HP	168.13	
	Laptop	HP	1209.59	
	Super Drive	HP	343.18	
	Router	HP	552.02	
	Monitor	Dell	252.01	
	Router	Dell	1061.15	
	Network Card	Dell	927.96	
	Super Drive	Dell	1450.74	
	Super Drive	Dell	1242.28	
	Router	Dell	1060.24	
	Desktop	Dell	1018.36	
	Monitor	Dell	1046.2	
	Router	Dell	1208.9	
	Network Card	Dell	522.32	
	Super Drive	Dell	534.35	
	Ethernet Ad…	Dell	516.77	
	Printer	Dell	983.49	
	Network Card	Dell	976.2	
	Network Card	Dell	361.22	
	Printer	Dell	1294.84	
	Hard Drive	Dell	970.45	
	Printer	Dell	398.11	
	Laptop	Dell	1069.61	

	Router	Dell	298.92	
	Laptop	Dell	1171.18	
	Hard Drive	Dell	355.71	
	Monitor	Dell	662.19	
	Super Drive	Dell	1033.91	
	Network Card	Dell	91.65	
	Router	Dell	351.24	
	Super Drive	Dell	104.17	
	Super Drive	Dell	488.54	
	Monitor	Leno…	720.05	
	Router	Leno…	882.78	
	Network Card	Leno…	800.89	
	Super Drive	Leno…	1029.24	
	Router	Leno…	945.18	
	Desktop	Leno…	980.5	
	Hard Drive	Leno…	975.81	
	Router	Leno…	554.02	
	Network Card	Leno…	1381.76	
	Super Drive	Leno…	1487.03	
	Ethernet Ad…	Leno…	126.82	
	Printer	Leno…	1347.2	
	Network Card	Leno…	1203.53	
	Network Card	Leno…	885.81	
	Printer	Leno…	866.69	
	Hard Drive	Leno…	903.48	
	Printer	Leno…	758.14	
	Laptop	Leno…	1370.48	
	Router	Leno…	796.89	
	Hard Drive	Leno…	645.44	
	Monitor	Leno…	375.59	
	Super Drive	Leno…	568.88	
	Network Card	Leno…	787.81	
	Router	Leno…	437.72	
	Super Drive	Leno…	61.84	
	Super Drive	Leno…	1298.03	

```sql
-- #6: List the annual total sales for each company (sort the results along the company and the year attributes).
select merchants.name as company, year(place.order_date) as rev_year, sum(sell.price) as total_sales
from merchants
join sell using (mid)
join products using (pid)
join contain using (pid)
join orders using (oid)
join place using (oid)
group by company, rev_year
order by company, rev_year;
-- Computes annual total sales per merchant by summing sell.price for each customer order grouped by merchant and year
```

company	rev_year	total_sales
Acer	2011	152986.29999999967
Acer	2016	60291.140000000014
Acer	2017	176722.7699999996
Acer	2018	262059.28999999954
Acer	2019	208815.79999999938
Acer	2020	182311.14999999944
Apple	2011	166822.9100000001
Apple	2016	64748.459999999955
Apple	2017	179560.77999999994
Apple	2018	300413.2299999994
Apple	2019	231573.17000000013
Apple	2020	216461.05999999988
Dell	2011	181730.35000000006
Dell	2016	71462.87
Dell	2017	182288.61000000007
Dell	2018	315004.82000000076
Dell	2019	221391.8300000004
Dell	2020	208063.08000000025
HP	2011	141030.14999999967
HP	2016	56986.120000000046
HP	2017	136092.4299999997
HP	2018	222707.07999999958
HP	2019	173334.0099999994
HP	2020	180775.17999999927
Lenovo	2011	184939.40999999997
Lenovo	2016	70131.56999999996
Lenovo	2017	197980.33000000013
Lenovo	2018	324291.5900000006
Lenovo	2019	232610.80000000028
Lenovo	2020	214154.2500000002

```sql
-- #7: Which company had the highest annual revenue and in what year?
select merchants.name as company, year(place.order_date) as rev_year, sum(sell.price)
from merchants
join sell using (mid)
join products using (pid)
join contain using (pid)
join orders using (oid)
join place using (oid)
group by company, rev_year
order by sum(sell.price) desc
limit 1;
```

23:87 | 2 errors found

Result Grid | Filter Rows: Search | Export: | Fetch rows:

company	rev_year	sum(sell.price)
Lenovo	2018	324291.5900000006

```
select merchants.name as company, year(place.order_date) as rev_year, sum(sell.price) as total_sales
from merchants
join sell using (mid)
join products using (pid)
join contain using (pid)
join orders using (oid)
join place using (oid)
group by company, rev_year
having total_sales >= all (
    select sum(sell.price)
    from merchants
    join sell using (mid)
    join products using (pid)
    join contain using (pid)
    join orders using (oid)
    join place using (oid)
    group by merchants.name, year(place.order_date)
);
-- Returns the merchant and year with the highest revenue by aggregating sales per merchant per year
-- and ordering by total revenue descending or a having block
```

52:110 2 errors found

Result Grid Filter Rows: Search Export:

company	rev_year	total_sales
Lenovo	2018	324291.5900000006

```
-- #8: On average, what was the cheapest shipping method used ever?
select shipping_method, avg(shipping_cost) as avg_cost
from orders
group by shipping_method
order by avg_cost
limit 1;
```

1:121 2 errors found

Result Grid Filter Rows: Search Export: Fetch rows:

shipping_meth...	avg_cost
USPS	7.455760869565214

```sql
select shipping_method, avg(shipping_cost) as avg_cost
from orders
group by shipping_method
having avg_cost <= all (
    select avg(shipping_cost)
    from orders
    group by shipping_method
);
-- Gives the shipping method with the lowest average cost by grouping orders by shipping_method and
-- selecting the minimum of avg(shipping_cost) using order by or a having block
```

100% 6:132 2 errors found

Result Grid Filter Rows: Search Export:

shipping_meth...	avg_cost
USPS	7.455760869565214

```sql
-- #9: What is the best sold ($) category for each company?
select merchants.name, category, sum(price) as total_sales
from contain
join products using (pid)
join sell using (pid)
join merchants using (mid)
group by merchants.name, products.category
having total_sales >= all (
    select sum(price)
    from contain
    join products using (pid)
    join sell using (pid)
    join merchants as merchants2 using (mid)
    where merchants2.name = merchants.name
    group by merchants.name, products.category
)
order by total_sales desc;
-- Determines each merchant's top-grossing product category by summing sell.price per merchant per category
-- and comparing each to all other categories of the same merchant
```

100% 1:153 2 errors found

Result Grid Filter Rows: Search Export:

name	category	total_sales
Acer	Peripheral	751705.6600000017
Apple	Peripheral	725401.4400000114
Lenovo	Peripheral	702791.9400000081
Dell	Peripheral	690326.4899999971
HP	Networking	446802.8700000033

```sql
-- #10: For each company find out which customers have spent the most and the least amounts.
select merchants.name, customers.fullname, sum(shipping_cost) as total_shipping_cost,
sum(price) as total_from_order, sum(shipping_cost) + sum(price) as total_cost
from customers
join place using (cid)
join orders using (oid)
join contain using (oid)
join products using (pid)
join sell using (pid)
join merchants using (mid)
group by merchants.name, customers.fullname
having total_from_order >= all (
    select sum(price)
    from customers
    join place using (cid)
    join orders using (oid)
    join contain using (oid)
    join products using (pid)
    join sell using (pid)
    join merchants as merchants2 using (mid)
    where merchants2.name = merchants.name
    group by merchants.name, customers.fullname
) or total_from_order <= all (
    select sum(price)
    from customers
    join place using (cid)
    join orders using (oid)
    join contain using (oid)
    join products using (pid)
    join sell using (pid)
    join merchants as merchants2 using (mid)
    where merchants2.name = merchants.name
    group by merchants.name, customers.fullname
);
-- Identifies the highest and lowest spending customers per merchant by summing their order totals and
-- using two having blocks, one for max and one for min.
```

name	fullname	total_shipping_cost	total_from_order	total_cost
Acer	Dean Heath	700.5400000000001	75230.29	75930.82999999999
Acer	Inez Long	330.8699999999999	31901.020000000004	32231.890000000003
HP	Clementine Travis	583.75	66628.06000000001	67211.81000000001
HP	Inez Long	305.5899999999998	26062.889999999996	26368.479999999996
Dell	Clementine Travis	633.5799999999998	85611.55	86245.13
Dell	Inez Long	348.42999999999984	31135.740000000005	31484.170000000006
Lenovo	Haviva Stewart	653.2499999999999	83030.25999999997	83683.50999999997
Lenovo	Inez Long	325.5899999999998	33948.91	34274.5

#5

- Assumptions
    - There is no functionality to display the exact price because contains is only linked to a product. This product can be sold by different merchants at different prices.

        - So my query displays all the merchants that sell a product Uriel bought, along with their prices

#6

- Assumptions
    - I believe this may overcount if the product is sold by more than one merchant, but I am unsure how to account for this

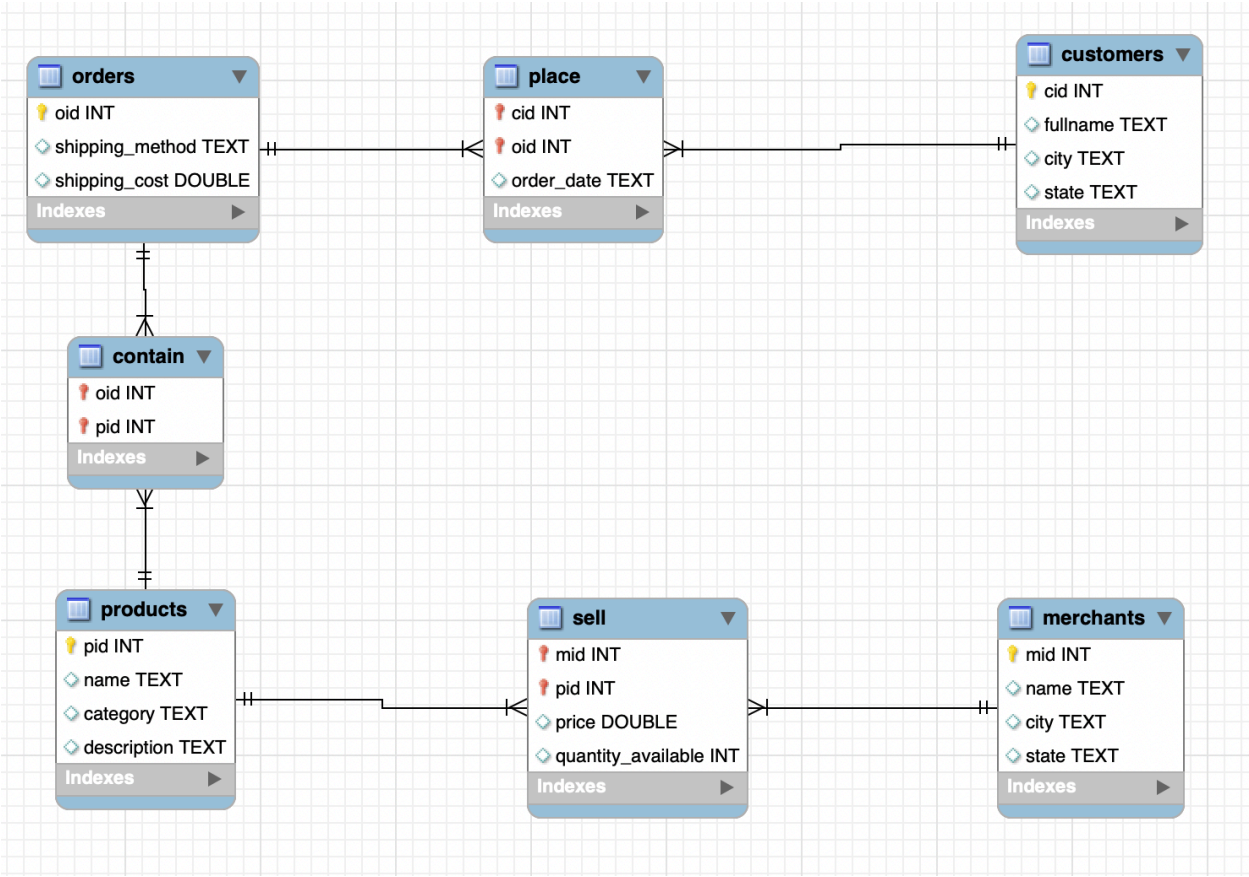
#9

- Assumptions
    - I believe this may overcount if the product is sold by more than one merchant, but I am unsure how to account for this

#10

- Assumptions
    - I believe this may overcount if the product is sold by more than one merchant, but I am unsure how to account for this
    - I also believe that the shipping_cost gets overcounted, but I am unsure how to account for this. I thought of using distinct, but that creates the opposite problem (too little cost)
    - Since I use a non-aggregate for the total spent by a customer, I don't know how to organize this for the highest and lowest customer at each company. If it was an aggregate, it would be easy to isolate the highest and lowest from each company. So I did this for the total_from_order column, although there may be some differences between that and the total_cost column.
        - When I did this, I got the customers for every company except Apple, so I am not sure what happened
        - I also noticed that this took a few seconds to run, so i wonder how to do this more efficiently

Primary keys and foreign keys shown here:



I didn't screenshot my SQL code for creating the constraints, so here are the results when I run show create table table_name;

contain, CREATE TABLE `contain` (
  `oid` int NOT NULL,
  `pid` int NOT NULL,
  PRIMARY KEY (`oid`,`pid`),
  KEY `fk_pid_contain` (`pid`),
  CONSTRAINT `fk_oid` FOREIGN KEY (`oid`) REFERENCES `orders` (`oid`),
  CONSTRAINT `fk_pid_contain` FOREIGN KEY (`pid`) REFERENCES `products` (`pid`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

customers, CREATE TABLE `customers` (
  `cid` int NOT NULL,
  `fullname` text,
  `city` text,
  `state` text,
  PRIMARY KEY (`cid`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

```
merchants, CREATE TABLE `merchants` (
  `mid` int NOT NULL,
  `name` text,
  `city` text,
  `state` text,
  PRIMARY KEY (`mid`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

orders, CREATE TABLE `orders` (
  `oid` int NOT NULL,
  `shipping_method` text,
  `shipping_cost` double DEFAULT NULL,
  PRIMARY KEY (`oid`),
  CONSTRAINT `check_ship_cost` CHECK (((`shipping_cost` >= 0) and (`shipping_cost` <=
500))),
  CONSTRAINT `check_ship_meth` CHECK ((`shipping_method` in
(_utf8mb4'UPS',_utf8mb4'FedEx',_utf8mb4'USPS')))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

place, CREATE TABLE `place` (
  `cid` int NOT NULL,
  `oid` int NOT NULL,
  `order_date` text,
  PRIMARY KEY (`cid`,`oid`),
  KEY `fk_oid_place` (`oid`),
  CONSTRAINT `fk_cid` FOREIGN KEY (`cid`) REFERENCES `customers` (`cid`),
  CONSTRAINT `fk_oid_place` FOREIGN KEY (`oid`) REFERENCES `orders` (`oid`),
  CONSTRAINT `valid_dates` CHECK ((`order_date` between _utf8mb4'1900-01-01' and
_utf8mb4'2025-12-31'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

products, CREATE TABLE `products` (
  `pid` int NOT NULL,
  `name` text,
  `category` text,
  `description` text,
  PRIMARY KEY (`pid`),
  CONSTRAINT `check_category` CHECK ((`category` in
(_utf8mb4'Peripheral',_utf8mb4'Networking',_utf8mb4'Computer'))),
  CONSTRAINT `check_name` CHECK ((`name` in (_utf8mb4'Printer',_utf8mb4'Ethernet
Adapter',_utf8mb4'Desktop',_utf8mb4'Hard
Drive',_utf8mb4'Laptop',_utf8mb4'Router',_utf8mb4'Network Card',_utf8mb4'Super
Drive',_utf8mb4'Monitor')))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

sell, CREATE TABLE `sell` (
  `mid` int NOT NULL,
  `pid` int NOT NULL,
  `price` double DEFAULT NULL,
  `quantity_available` int DEFAULT NULL,
  PRIMARY KEY (`mid`,`pid`),
  KEY `fk_pid` (`pid`),
  CONSTRAINT `fk_mid` FOREIGN KEY (`mid`) REFERENCES `merchants` (`mid`),
  CONSTRAINT `fk_pid` FOREIGN KEY (`pid`) REFERENCES `products` (`pid`),
  CONSTRAINT `check_price_range` CHECK (((`price` >= 0) and (`price` <= 100000))),
  CONSTRAINT `check_quant_avail` CHECK (((`quantity_available` >= 0) and
(`quantity_available` <= 1000)))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci